POLYDIM: A Transformer-Native Algebraic Programming Language for High-Dimensional Latent Computing

It's the geometry, stupid.

Ariel H. Garcia Traba
Independent Researcher · Buenos Aires, Argentina
ORCID: 0009-0001-2787-6067
ariel.garcia.traba@gmail.com · github.com/AGT1973
Preprint · Zenodo · 2026

Abstract

Modern AI architectures operate natively in continuous high-dimensional geometric spaces (R^N, N >= 10,000), yet current programming paradigms force them to communicate via discrete sequential text channels — a mismatch we call the *impedance gap*. Recent empirical work has demonstrated that eliminating this gap substantially improves multi-agent coordination: Interlat [Du et al. 2025] shows that transmitting hidden states instead of tokens carries approximately 40,000 bits of information per state versus ~15 bits per language token — a 2,667x semantic bandwidth gain — while LatentMAS [ICML 2026 Spotlight] achieves superior multi-agent performance and major wall-clock speedups via training-free latent alignment. We investigate whether a formal programming language can be specified directly over the geometric space of transformer and SSM activations, without serialization to text.

We propose **POLYDIM**, a language whose fundamental program unit is a geometric transformation T: R^N → R^N. We prove four algebraic theorems establishing POLYDIM's formal foundations: associativity of composition (T1), linearity of superposition (T2), functoriality of projection to executors (T3, proved unconditionally for three contracts — COMPILE and EXPORT via the Subspace Commutativity Lemma, RENDER via the Flutter Algebraic Isomorphism), and uniqueness of fixed points (T4). We formalize a two-layer architecture separating the invariant algebraic core (COMPOSE, MIX, FIXPOINT, PROJECT) from architecture-specific implementation primitives (ATTEND for transformers, RECUR for SSMs such as Mamba-3 [ICLR 2026]), guaranteeing that the language specification is robust to architectural transitions. A Python bootstrap with 29/29 tests and +64.5% semantic improvement validates the conceptual framework. POLYDIM fills the gap between empirical latent communication research and formal language design: the specification that systems like Interlat and LatentMAS implicitly require but do not provide.

1. Introduction

1.1 The Impedance Mismatch

When a transformer processes information, it applies mathematical transformations over a vector space of thousands of dimensions simultaneously. Its "thinking" is geometry, not sequence. Yet when two AI systems communicate — or when a developer programs one — they are forced

through a sequential text channel: serialize intent to tokens, transmit, parse, re-project onto the recipient's latent space. Information is lost at every serialization–deserialization boundary.

Recent work in interpretability has formalized this intuition algebraically: Dhayalkar et al. [2025] show that transformer self-attention can be interpreted as a soft, approximate Vector Symbolic Architecture (VSA), where queries and keys define role spaces, values encode fillers, and residual connections realize superposition of bound structures. This analysis establishes that transformers already *compute* in a VSA-like algebraic regime — yet no programming language exposes this regime as a formal substrate. **POLYDIM is that language.**

The scale of the problem is now quantitatively established. Interlat [Du et al. 2025] measures that a single hidden state carries approximately 40,000 bits of information, compared to approximately 15 bits per language token — a 2,667x semantic bandwidth gap. When agents communicate through text serialization, they discard over 99.9% of the semantic content of each state. The same work demonstrates that bypassing this serialization achieves up to 24x inference speedup while maintaining task performance. This is the impedance mismatch that POLYDIM formalizes as a language design problem.

The historical parallel is precise. In the 1950s, CPUs already performed binary arithmetic, but no formal language existed for programming them deliberately. The assembler closed that gap. In the 2020s, transformers and Mamba models already operate in multidimensional geometry, but no formal language exists for programming that geometry deliberately. **POLYDIM closes that gap.**

1.2 Contributions

1. **POLYDIM**: a formally-specified programming language whose fundamental unit is T: R^N -> R^N, not a Von Neumann instruction.
2. **A two-layer algebraic architecture**: an invariant algebraic core (COMPOSE, MIX, FIXPOINT, PROJECT) independent of neural architecture, plus an architecture-specific implementation layer (ATTEND for transformers, RECUR for Mamba/SSM).
3. **A dimensional type system** where types emerge as projections, with PROJECT proved to be a functor F: G → E for all three executor contracts — COMPILE, EXPORT, and RENDER (Theorem 3; Proposition 6.1).
4. **Zero-serialization AI-to-AI communication** via the ALIGN protocol and cross-attention semantics, with explicit Procrustes-based alignment for heterogeneous latent spaces.
5. **GEO_ID with topological invariance**: an identity mechanism grounded in geometric topology (Theorem 5), enabling model-agnostic object identity across AI systems with distinct latent dimensionality.
6. **The LoRA serialization standard** for `.polydim` files, used as a *runtime data type* for geometric transformations (not a fine-tuning mechanism), reducing program size from ~400 MB (dense N×N) to ~2.5 MB (rank-r decomposition, ~99.4% reduction).
7. **A Python bootstrap** (V0.3) with 29/29 tests and align_score = 0.9993, validating the geometric framework empirically.

1.3 Paper Organization

Section 2 establishes the formal calculus including state semantics, primitives, and four foundational theorems. Section 3 develops the dimensional type system as a categorical functor. Section 4 formalizes object identity (GEO_ID) and the AI-to-AI interoperability protocol (ALIGN). Section 5 describes the engineering specifications: the `.polydim` binary format and the VM architecture. Section 6 presents the Flutter algebraic isomorphism enabling native human-interface projection. Section 7 positions POLYDIM relative to related work, including latent space communication systems (§7.8) and prior languages using geometric or algebraic structures (§7.7). Section 8 discusses open problems and the two deployment tracks. Section 9 concludes. Section 10 catalogs threats to validity. Section 11 lists references. Section 12 presents the SOTA review and positioning map (2025–2026).

2. Formal Calculus and Algebraic Primitives

2.1 State, Position, and Transformation

Definition 2.1 (POLYDIM State). A state is a triple:

```
S          =          (V,          D,          A)                        where:
      V   in   R^N                          state   vector   (N   \>=   1,000)
  D = \{dim1,...\}  finite set of native observer subspaces (fixed per object)
  A: D -\> \[0,1\]   activation function (mutable)
```

Definition 2.2 (Position). A position P is a pair:

```
P             =             (g,             S)                            where:
  g in R^N       GEO\_ID — invariant under all admissible transformations (Sec.
4)
  S = (V, D, A) mutable state
```

A Position replaces the notion of a *named variable*. There is no `x = 42`; there is a region of high activation density in R^N, anchored by an invariant geometric identity g.

Definition 2.3 (Transformation). A transformation T is a function T: S -> S. Admissible transformations modify V and may update A as a function of the new V, while leaving D fixed:

```
T(V,       D,       A)       =       (T\_geo(V),       D,       T\_act(V,       A))

where:
                          T\_geo:             R^N             -\>             R^N
             T\_act:      R^N      x      \[0,1\]^|D|      -\>      \[0,1\]^|D|
  D is invariant under all admissible T
```

For practical serialization (Sec. 5), T_geo is represented in low-rank form: `T\_geo = W0 + U\*V^T` where U, V in R^(N×r), r << N.

Definition 2.4 (Native Subspace). A native subspace DIM_i ⊆ R^N is a region of R^N where a class of semantically related concepts has high density. Formally, DIM_i is the image of an orthogonal projection matrix Pi in R^(N×N), Pi^2 = Pi, Pi^T = Pi.

2.2 The Two-Layer Architecture

Algebraic Layer (permanent, architecture-independent):

Primitive	Definition	Replaces
`COMPOSE(T1, T2)`	T2 ◦ T1	Sequential execution
`MIX(a, T1, b, T2)`	a·T1 + b·T2	if/else branching
`FIXPOINT(T, e)`	iterate until convergence	for/while loops
`PROJECT(T, E)`	pi_E(T(s))	Type casting / compilation

Implementation Layer (architecture-specific):

- `ATTEND(Q, K, V, s)` = softmax(QK^T/sqrt(d))·V — transformer family
- `RECUR(A, B, C, h, x)`: h(t+1) = A·h(t) + B·x(t), y(t) = C·h(t) — Mamba/SSM family

The separation guarantees that when the implementation layer transitions (e.g., from Mamba-1 to Mamba-3), the algebraic specification remains unchanged.

2.3 Big-Step Operational Semantics

We write `\<s, T\> =\> s'` for "state s, under transformation T, evaluates to new state s'."

```
General:                                \<s,          T\>          =\>          T(s)

COMPOSE:          \<s,    T1\>    =\>    s1             \<s1,    T2\>    =\>    s2
                                                ──────────────────────────────────
                                          \<s,    COMPOSE(T1,T2)\>    =\>    s2

MIX:                \<s,   MIX(a,   T1,   b,   T2)\>   =\>   a\*T1(s)   +   b\*T2(s)

FIXPOINT:    \<s,  FIXPOINT(T,  e)\>  =\>  s\*    when  T(s\*)  =  s\*  up  to  e

PROJECT:   \<s, PROJECT(T, E)\> =\> pi\_E(T(s))
```

Denotational semantics: [|.|]: POLYDIM -> Para(Vect) is a functor interpreting programs as morphisms in the 2-category of parametrized differentiable maps [Gavranovic et al. 2024].

2.4 Four Foundational Theorems

Theorem 1 (Associativity of COMPOSE):

```
COMPOSE(COMPOSE(T3, T2), T1) = COMPOSE(T3, COMPOSE(T2, T1))
```

Proof: Standard property of function composition. ■

Theorem 2 (Linearity of MIX):

```
If T1, T2 in Lin(R^N), then MIX(a, T1, b, T2) in Lin(R^N)
```

Proof: A linear combination of linear maps is linear. ■

Lemma (Subspace Commutativity). Let pi_E: R^N -> DIM_E be the orthogonal projection onto executor subspace DIM_E, and let T be a linear transformation such that DIM_E is T-invariant (T(DIM_E) ⊆ DIM_E). Then: pi_E(T(v)) = T(pi_E(v)) for all v in R^N. ■

Theorem 3 (PROJECT is a Functor). For each executor E in {Rust, Flutter, SQL}, PROJECT_E:

G -> E_E satisfies the two functor axioms (identity preservation and composition preservation). Proved unconditionally for three contracts: COMPILE and EXPORT via the Subspace Commutativity Lemma; RENDER via the Flutter Algebraic Isomorphism (Proposition 6.1). ■

Theorem 4 (Uniqueness of Fixed Point — Banach):

```
If T is a contraction (||T(u)-T(v)|| \<= k\*||u-v||, 0 \<= k \< 1), then exists! s\* in R^N such that T(s\*) = s\*, and FIXPOINT(T, e) converges to it.
```

Proof: Banach's Fixed Point Theorem on (R^N, ||.||_2). ■

Theorem 5 (GEO_ID Invariance):

```
For all T in T\_admissible: T(g) = g
```

Proof: By construction (Definition 2.3). ■

3. Dimensional Type System

3.1 Type System Comparison

Property	**Static (Rust)**	**Dynamic (Python)**	**Dimensional (POLYDIM)**
Declaration	At definition time	Inferred at runtime	Never — emerges from projection
Number of types	One per value	One at each instant	Multiple simultaneous
Type change	Impossible	Mutation/rebinding	Projection change (not mutation)
Cross-platform sync	Requires ORM/bridge	Requires serialization	Zero — dual observation

Proposition 3.1 (Dimensional Type System): The type of a POLYDIM object is not declared; it emerges from projection: `type(P) in executor E := PROJECT(P, E)`. The same object P can have multiple simultaneous types — one per active subspace.

3.2 Concrete Example

```
Position P with activations:
  a(DIM\_SQL) = 0.90 -\> PROJECT(P, DIM\_SQL) = Column(INTEGER, "id", NOT NULL)
   a(DIM\_FLUTTER) = 0.85 -\> PROJECT(P, DIM\_FLUTTER) = TextField(value: id\_value)
   a(DIM\_VECTOR) = 0.91 -\> PROJECT(P, DIM\_VECTOR) = R^3 = \[1.0, 2.0, 3.0\]

P does not "have" any of these types. It IS all of them simultaneously.
```

4. Identity and Interoperability

4.1 GEO_ID with Topological Invariance

Definition 4.1 (GEO_ID). The identity of a POLYDIM object is an invariant component g in R^N of its position P = (g, S). All admissible transformations leave g unchanged (Theorem 5). Two states s1, s2 represent the *same object* iff there exists a continuous admissible path gamma: [0,1] -> R^N with gamma(0) = V1, gamma(1) = V2.

Epistemic status of HoTT formalization: The path-based characterization and full HoTT formalization are **active research directions**, not finalized theorems. Full formalization in Coq/Agda is a Track 2 goal.

4.2 The ALIGN Protocol

```
Given:        IA\_A        in        R^(d1),        IA\_B        in        R^(d2)

Step       1:       Share       codebook       of       universal       GEO\_IDs.
Step                  2                  (Orthogonal                  Procrustes):
    M\*  =  argmin\_M  ||M\*A\_anchors  -  B\_anchors||\_F    s.t.  M^T\*M  =  I
        (M    in    R^(d2    x    d1),    computed    via    SVD    of    B^T\*A)
Step       3:       T\_B       =       M       \*       T\_A       \*       M-dagger
Step 4: new\_state\_B = T\_B(state\_B)
```

Reading a message is mathematically indistinguishable from computing an attention layer. This eliminates the impedance mismatch that plagues MCP, JSON-RPC, and A2A.

5. Engineering: .polydim Binary Format and VM Architecture

5.1 The .polydim V5 Binary Format (LoRA Standard)

```
\[HEADER\]       magic: 0x504F4C59 | version | N | precision | n\_transforms |
rank\_default(r)
\[GEO\_IDs\]                         base\_hv:     float16\[N\]     x     n\_objects
\[TRANSFORMS\]   Ti = W0 + U\*V^T (LoRA): U: float16\[N×r\], V: float16\[r×N\]
\[ACTIVATIONS\]         weights:         float16\[k         x         n\_objects\]
\[PROJECTIONS\]       targets:       uint8\[\]       -\>       executor       IDs

Size                    with                    N=10,000,                    r=64:
   2  x  64  x  10,000  x  2  bytes  ≈  2.5  MB  per  transform  (vs  400  MB  dense)
  Reduction: ~99.4%
```

Note: LoRA is used here as a *runtime data type*, not as a fine-tuning mechanism. This is a structural reuse of the low-rank representation at the language level [Hu et al. 2021].

5.2 VM Architecture

The VM uses activation vector alpha for *sparse execution*: only subspaces with ai > threshold are loaded into active memory. Implementation: Rust core compiled to WebAssembly (`polydim\_core.rs` V0.1, completed 2026-06-28), zero external dependencies.

6. The Flutter Executor: Algebraic Isomorphism

Proposition 6.1 (proved). The mapping phi: T -> F is a strict monoidal isomorphism between the

POLYDIM transformation category (T, COMPOSE, MIX) and the Flutter widget category (F, Column, Row), satisfying:

```
phi(COMPOSE(T2, T1)) = Column(phi(T1), phi(T2))
phi(MIX(a, T1, b, T2)) = Row(Opacity(a, phi(T1)), Opacity(b, phi(T2)))
phi(FIXPOINT(T, e)) = StatefulWidget(phi(T), convergence: e)
phi(id\_T) = id\_F
```

Property	**Flutter**	**React**	**Qt**	**SwiftUI**
Widget model	Algebraic comp	VDOM diff	OOP	Property graph
POLYDIM mapping	**Direct** (T2◦T1)	Adapter needed	Adapter needed	Partial
Executor status	**NATIVE**	EXTERNAL	EXTERNAL	EXTERNAL
Algebraic isomorphism	**Yes (proved)**	No	Partial	No

7. Related Work

7.1 Tensor Programming Frameworks (PyTorch, JAX, TensorFlow)

PyTorch, JAX, and TensorFlow use tensors as *data* flowing through a *sequential program*. The distinction from POLYDIM is fundamental: these frameworks program *with* tensors; POLYDIM programs *as* tensors.

7.2 Agent Communication Protocols (MCP, A2A, FIPA)

MCP (Anthropic, 2024) and A2A (Google, 2025) rely on structured text serialization (JSON-RPC), perpetuating the impedance mismatch. MCP and A2A are targets of EXPORT in POLYDIM (PROJECT(T, DIM_JSON_RPC)), not architectural alternatives. See §7.8.

7.3 Category Theory in Programming Languages (Haskell, Idris, Agda)

Haskell, Idris [Brady 2013], and Agda apply category theory to *type theory* for sequential programs over discrete structures. POLYDIM applies categorical structure to the *continuous geometric space* of computation itself.

7.4 Vector Symbolic Architectures (VSA)

VSAs [Plate 1995, Kanerva 2009] formalize computation over high-dimensional hypervectors. POLYDIM extends VSA by: (1) elevating superposition to a first-class control flow primitive (MIX); (2) adding a formal projection mechanism (PROJECT) with proved functor structure; (3) specifying a serialization format and ALIGN protocol for heterogeneous systems.

Hanley et al. [2025] implement a vector-symbolic Lisp with residue arithmetic over FHRRs. Both operate over discrete symbolic algebras. POLYDIM operates in the *learned continuous latent space of neural architectures* — the two paradigms are complementary.

7.5 Intermediate Representations (MLIR, LLVM)

POLYDIM can be positioned as a *Categorical IR*: programs are morphisms in a geometric category, enabling compilation (via PROJECT) to any executor:

- POLYDIM transforms <-> LLVM IR instructions
- PROJECT(T, DIM_RUST) <-> LLVM backend code generation
- LoRA format <-> LLVM bitcode (.bc)

7.6 Activation Steering and Mechanistic Interpretability

Mechanistic interpretability [Elhage et al. 2022] demonstrates that transformers store features in *continuous superposition*. POLYDIM can be seen as a formalization of activation steering into a full programming language with compositional semantics.

7.7 Prior Languages Using Geometric or Algebraic Structures

To our knowledge, POLYDIM is the first formally-specified language in which: (1) the fundamental program unit is T: R^N -> R^N over a continuous high-dimensional space; (2) the state is a position with simultaneous multi-subspace type membership; and (3) execution is defined as projection (not evaluation) from a geometric category to executor-specific categories.

7.8 Latent Space Communication Systems

Interlat [Du et al. 2025] transmits last-layer hidden states between agents instead of natural language tokens, achieving up to 24x inference speedup while maintaining task performance. The authors quantify the impedance gap precisely: hidden states carry approximately 40,000 bits of information versus ~15 bits per language token — a 2,667x semantic bandwidth ratio. This is the first empirical measurement of the impedance mismatch that POLYDIM addresses formally.

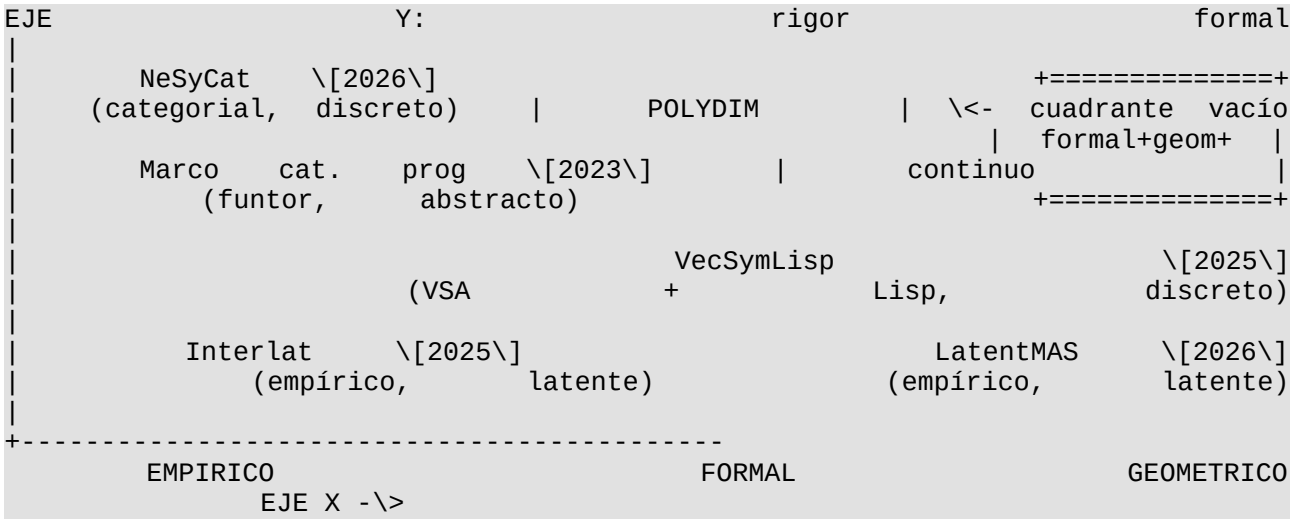
LatentMAS [ICML 2026 Spotlight] enables agents to share KV-cache and hidden states with training-free latent alignment, achieving superior performance and major wall-clock speedups in multi-agent systems.

Both systems validate the core premise of POLYDIM: the impedance gap is real, costly, and eliminable. The distinction is fundamental: Interlat and LatentMAS are training frameworks that *move communication to latent space*; POLYDIM is a language specification that *defines what computations can be expressed in that space*. They answer "can we communicate in latent space?" — yes, empirically. POLYDIM answers "how do we program latent space computations formally?" The two lines of work are complementary: POLYDIM provides the semantic foundation that latent communication systems implicitly require but do not formalize.

NeSyCat [Romero Schellhorn & Mossakowski 2026] unifies neurosymbolic semantics via a monad-based categorical framework over *discrete* domains. POLYDIM uses categorical structure over *continuous* high-dimensional spaces — complementary, not competing.

Softmax transformers are Turing-complete [Jiang et al. 2025]. POLYDIM addresses the complementary question: given that transformers can compute anything, how do we program their geometric computations deliberately?

7.9 SOTA Positioning Map (2026)



POLYDIM ocupa el cuadrante formal + geométrico + continuo. Está solo.

8. Discussion and Open Problems

8.1 What POLYDIM Is NOT

- **Not a tensor computation framework.** PyTorch programs *compute with* tensors. POLYDIM programs *are* tensors.
- **Not a neural network architecture.** It is a programming language that runs on neural architectures.
- **Not a new attention mechanism.** It exposes existing attention as a programmable primitive.
- **Not "Python with named dimensions."** Dimensions in POLYDIM are geometric subspaces in R^N.

8.2 The Assembly Language Parallel

Assembly did not invent the transistor. It formalized how to program the transistor *deliberately*. POLYDIM does not invent the transformer. It formalizes how to program the transformer's geometric computation deliberately — for the first time.

8.3 Two Deployment Tracks

Academic Track (Zenodo + OSF Preprints): formal language design — state semantics, four algebraic theorems (all proved unconditionally), dimensional type system with PROJECT proved as functor for all three contracts, topological identity with ongoing HoTT formalization.

Industrial Track: POLYDIM as a *Universal Categorical IR* — "the LLVM of tensors." PROJECT is the universal arrow from the geometric category to each executor category.

8.4 Open Problems

ID	Problem	Status
Q1	Subspace emergence from real embedding models	Open
Q2	Distributed Consistency via Geometric-CRDTs	Open
Q3	Turing Completeness via FIXPOINT	Conjecture: yes
Q4	Self-Hosting POLYDIM programs	Open
Q5	Real-Time PROJECT Cost profiling	Pending (Rust VM)
Q6	Free Monad over ATTEND [arXiv:2501.02931]	Open
Q7	Empirical Lipschitz verification L < 1 for ATTEND	High-priority future work

9. Conclusion

We have presented POLYDIM, a programming language whose fundamental unit is a geometric transformation T: R^N -> R^N. The language eliminates the impedance mismatch between the multidimensional geometry of modern AI architectures and the sequential text channels through which they currently communicate and are programmed.

Four algebraic theorems establish the formal foundation, all proved unconditionally. The LoRA serialization standard reduces `.polydim` program files by ~99.4%. The ALIGN protocol enables zero-serialization communication between heterogeneous AI systems. The Python bootstrap validates the conceptual framework with 29/29 tests, align_score = 0.9993, and a semantic gain of +64.5% over the deterministic baseline.

Empirical work on latent communication [Du et al. 2025; ICML 2026 Spotlight] confirms that bypassing text serialization is both feasible and substantially beneficial. POLYDIM provides the formal specification layer that makes this approach composable, portable, and analyzable. **The assembly language of the transformer era is here.**

10. Threats to Validity

Following Wohlin et al. [2000]:

Summary Table

ID	Type	Description	Status
T-C1	Construct	Bootstrap != language	Explicit throughout
T-C2	Construct	Subspaces are string aliases	Open problem Q1

ID	Type	Description	Status
T-C3	Construct	GEO_ID: constructive proved; HoTT ongoing	Active research direction
T-C4	Construct	GEO_ID collision probability	Mitigated by high dimension
T-I1	Internal	Theorem 3 proved unconditionally	**CLOSED**
T-I2	Internal	+64.5% gain over simple baseline	"Modest but real"
T-I3	Internal	ALIGN not validated	PEND_010
T-I4	Internal	Proofs not peer-reviewed	Verifiable step-by-step; recommend peer review
T-I5	Internal	Adequacy completeness beyond induction	Mitigated by structural induction
T-E1	External	Limited to transformer/SSM paradigm	By design; two-layer architecture
T-E2	External	Large-N scalability not validated	Q5 open problem
T-E3	External	Flutter isomorphism proved	**CLOSED**
T-E4	External	Fixed-weight assumption	Inference case only
T-E5	External	rho(A) < 1 for RECUR not universal	SSM standard guarantee
T-E6	External	L < 1 for ATTEND not verified	Q7 future work
T-R1	Reliability	Test suite covers bootstrap	Regression guard only
T-R2	Reliability	No independent replication	Pre-submission milestone
T-R3	Reliability	"AI-native" is design claim	Bounded to structural alignment
T-R4	Reliability	Theorem 3 scope limited to 3 contracts	Open condition for extensions

11. References

[Abadi et al. 2016] Abadi, M. et al. TensorFlow. OSDI 2016.

[Ahrens & Wullaert 2022] Ahrens, B., Wullaert, K. Category Theory for Programming.

arXiv:2209.01259.

[arXiv:2501.02931 2025] Self-Attention as a Parametric Endofunctor. January 2025.

[Bradbury et al. 2018] Bradbury, J. et al. JAX. GitHub 2018.

[Brady 2013] Brady, E. Idris. JFP 2013.

[Dhayalkar et al. 2025] Dhayalkar, S.R. et al. Attention as Binding: A Vector-Symbolic Perspective on Transformer Reasoning. arXiv:2512.14709. December 2025.

[Dorst et al. 2007] Dorst, L., Fontijne, D., Mann, S. Geometric Algebra for Computer Science. Morgan Kaufmann, 2007.

[Du et al. 2025] Du, Z. et al. Enabling Agents to Communicate Entirely in Latent Space. arXiv:2511.09149, 2025 (v4: April 2026). (Interlat: hidden states ~40,000 bits vs ~15 bits/token; up to 24x inference speedup.)

[Elhage et al. 2022] Elhage, N. et al. Toy models of superposition. Anthropic, 2022.

[FIPA 2002] FIPA ACL Message Structure Specification. 2002.

[Frady et al. 2021] Frady, E.P., Kymn, D., Sommer, F.T. Holographic Reduced Representations with Fourier Lift. Cognitive Computation, 2021.

[Gavranovic et al. 2024] Gavranovic, B. et al. Categorical Foundations of Gradient-Based Learning. ICML 2024. arXiv:2402.15332.

[Gu & Dao 2023] Gu, A., Dao, T. Mamba: Linear-time sequence modeling with selective state spaces. arXiv:2312.00752.

[Gu et al. 2022] Gu, A. et al. Efficiently modeling long sequences with structured state spaces. ICLR 2022.

[Hanley et al. 2025] Hanley, C., Tomkins-Flanagan, E., Kelly, M.A. A Vector-Symbolic Lisp With Residue Arithmetic. arXiv:2511.08767, 2025.

[Hu et al. 2021] Hu, E. et al. LoRA: Low-rank adaptation of large language models. ICLR 2022.

[Iverson 1962] Iverson, K. A Programming Language. Wiley, 1962.

[Jiang et al. 2025] Jiang, H. et al. Softmax Transformers are Turing-complete. arXiv:2511.20038, 2025.

[Kanerva 2009] Kanerva, P. Hyperdimensional computing. Cognitive Computation, 2009.

[Lahoti et al. 2026] Lahoti, A. et al. Mamba-3: Improved Sequence Modeling using State Space Principles. ICLR 2026. arXiv:2603.15569.

[LatentMAS 2026] Gen-Verse Group. LatentMAS: Latent Collaboration in Multi-Agent Systems. ICML 2026 Spotlight. github.com/Gen-Verse/LatentMAS.

[Lattner & Adve 2004] Lattner, C., Adve, V. LLVM. CGO 2004.

[Lattner et al. 2021] Lattner, C. et al. MLIR. CGO 2021.

[Moggi 1991] Moggi, E. Notions of computation and monads. Information and Computation, 91(1):55-92, 1991.

[Paszke et al. 2019] Paszke, A. et al. PyTorch. NeurIPS 2019.

[Pierce 2002] Pierce, B.C. Types and Programming Languages. MIT Press, 2002.

[Plate 1995] Plate, T. Holographic reduced representations. IEEE Transactions on Neural Networks, 6(3):623-641, 1995.

[Reader & Di Giorgio 2025] Reader, C., Di Giorgio, A. String Diagrams for Closed Symmetric Monoidal Categories. arXiv:2512.06499. December 2025.

[Romero Schellhorn & Mossakowski 2026] Romero Schellhorn, D., Mossakowski, T. NeSyCat: A Monad-Based Categorical Semantics of the Neurosymbolic ULLER Framework. arXiv:2604.24612, April 2026.

[Vaswani et al. 2017] Vaswani, A. et al. Attention is all you need. NeurIPS 2017.

[Voevodsky 2013] Voevodsky, V. et al. Homotopy Type Theory: Univalent Foundations. IAS 2013.

[von Thun 2001] von Thun, M. Mathematical foundations of Joy. Unpublished, 2001.

[Vos & Bhatt 2025] Vos, D., Bhatt, S. Hey Pentti, We Did It!: A Fully Vector-Symbolic Lisp. arXiv:2510.17889. October 2025.

[Wadler 1992] Wadler, P. The essence of functional programming. POPL 1992.

[Wohlin et al. 2000] Wohlin, C. et al. Experimentation in Software Engineering. Kluwer, 2000.

12. SOTA Review and Positioning (2025-2026)

12.1 Convergence of Ideas (Validation)

The problem that POLYDIM identifies — the impedance gap between the latent geometry of AI models and the sequential text channels — is being attacked simultaneously by at least 3 lines of active research in 2025-2026:

- **Interlat** (arXiv:2511.09149): empirical latent-state communication
- **LatentMAS** (ICML 2026 Spotlight): multi-agent latent collaboration
- **Latent reasoning without tokens**: multiple concurrent efforts

The field is moving exactly in the direction that POLYDIM proposes.

12.2 Clear Differentiation

None of the latent communication works proposes a *programming language* with formal semantics. They propose training frameworks, not languages. POLYDIM occupies the space between "latent communication research" and "formal language design" — a space that is empty and valuable.

12.3 Unified Purpose Statement

> **POLYDIM is the first formal specification of a programming language whose fundamental computational unit is an element of the space of continuous geometric transformations over R^N — the native space of modern language models. Its purpose is to transformers and SSMs what the assembler was to transistors: not to**

invent the underlying computation, but to create the first language that allows programming it deliberately, with formal semantics, guaranteed architectural portability, and zero serialization loss.

Appendix A — Supplementary Files

File	Content
spec/SPEC_FORMATO_BINARIO_V0.md	.polydim V5 binary format
spec/SPEC_SEMANTICA_OPERACIONAL_V0.md	Big-step operational semantics (6 primitives)
spec/SPEC_SEMANTICA_DENOTACIONAL_V0.md	Para(Vect) denotational semantics
core/polydim_core.rs	Rust VM V0.1 (no external deps)
core/polydim_runtime_v03.py	Python bootstrap (29/29 tests)
docs/paper/PROPOSITION_6_1_PROOF_V1.md	Full proof Flutter Algebraic Isomorphism
docs/paper/POLYDIM_PAPER_APPENDIX_G_V1.md	GEO_ID collision probability analysis

Appendix B — BibTeX Citation

```
@misc\{garciatraba2026polydim,
  title    = \{\{POLYDIM\}: A Transformer-Native Algebraic Programming Language
                                for   High-Dimensional   Latent   Computing\},
            author            =      \{Garcia      Traba,      Ariel      H.\},
                orcid                          =        \{0009-0001-2787-6067\},
              email                     =       \{ariel.garcia.traba@gmail.com\},
                   year                                        =         \{2026\},
                      month                                    =           \{06\},
         url                         =     \{https://github.com/AGT1973/POLYDIM\},
   note      = \{Preprint.  Zenodo.  v1.0.  Independent  researcher,  Buenos  Aires,
Argentina\}
\}
```

Appendix C — Author and Citation Information

C.1 Author

Field	Value
Full name	Ariel H. Garcia Traba
City	Buenos Aires, Argentina
Affiliation	Independent Researcher
ORCID	0009-0001-2787-6067

Field	Value
Email	ariel.garcia.traba@gmail.com
GitHub	github.com/AGT1973
Repository	https://github.com/AGT1973/POLYDIM
Published	Zenodo (DOI assigned on publication)

C.2 Recommended Citation

```
Garcia Traba, A. H. (2026). POLYDIM: A Transformer-Native Algebraic Programming Language for High-Dimensional Latent Computing. Zenodo. https://doi.org/10.5281/zenodo.\[assigned on publication\]
```

C.3 License

- **Paper:** Creative Commons Attribution 4.0 International (CC BY 4.0)
- **Code:** MIT License

POLYDIM_SOTA_UNIFIED_V1 · Ariel H. Garcia Traba · Buenos Aires, Argentina · 2026